

Secure Coding Review

Vulnerability Assessment of a Python/Flask Web Application — "VulnBank"

Intern Name	Mohamed unaiz
Internship	Code Alpha — Cybersecurity Internship
Task	Task 3 of 4 — Secure Coding Review
Target Application	VulnBank (Python 3 / Flask, intentionally vulnerable)
Review Method	Manual source code audit + live exploitation testing
Report Date	2026-06-16
Repository	Secure Coding Review

Submission Summary

This report documents a manual secure code review and live exploitation test of "VulnBank", a deliberately vulnerable Python/Flask demo application. Ten vulnerabilities spanning SQL Injection, Stored XSS, IDOR, and Broken Access Control were identified, documented with proof-of-concept exploits, and confirmed through live testing. Each finding includes impact analysis and code-level remediation.

1. Objective

The goal of Task 3 was to select a programming language and application, perform a code review to identify security vulnerabilities using static analysis and manual inspection methods, and document findings with remediation steps for safer code.

Specific Goals

- Select a programming language and application to audit.
 - Perform a code review to identify security vulnerabilities.
-

- Use tools like static analyzers or manual inspection methods.
- Provide recommendations and best practices for secure coding.
- Document findings and suggest remediation steps for safer code.

2. Tools & Technologies Used

Component	Technology	Purpose
Target App	Python 3 / Flask	Web application under review ("VulnBank")
Database	SQLite3	Backend data store for users and comments
Review Method	Manual code inspection	Line-by-line review of all routes and templates
Exploitation	Browser + URL manipulation	Live testing of SQLi, IDOR, and access control
Environment	Kali Linux + Firefox	Testing platform (10.235.124.114:5000)

3. Methodology

VulnBank is a small Flask application with six routes: home, login, comments, admin panel, per-user profile endpoint, and file download. The review combined manual static analysis of Flask route handlers (reading source line-by-line for unsafe patterns) with dynamic testing (interacting through a browser to confirm exploitability).

3.1 Review Scope

Component	In Scope	Review Method
Login / Auth	Yes	Manual review + SQL injection testing
Comment System	Yes	Stored XSS payload review
File Download	Yes	Path traversal code review

Component	In Scope	Review Method
Admin Panel	Yes	Access control verification
User Profile API	Yes	IDOR testing — live exploitation
Session Handling	Yes	Cookie configuration review

3.2 Application Setup

```
# Launch the vulnerable target application
pip install flask
python3 task3_vulnerable_app.py
# App starts on http://0.0.0.0:5000
# Auto-creates SQLite DB at /tmp/vuln_app.db
# Default credentials: admin/admin and alice/alice
```

4. Live Execution & Findings

The application was launched and reviewed live in a Kali Linux browser session at <http://10.235.124.114:5000>. Each screenshot below is annotated against the corresponding vulnerability finding.

4.1 Application Home Page — Finding V-06 (Broken Access Control)

The home page greets the user as "Guest" and exposes a direct link to the /admin panel in the navigation bar — accessible without any authentication, confirming Finding V-06.

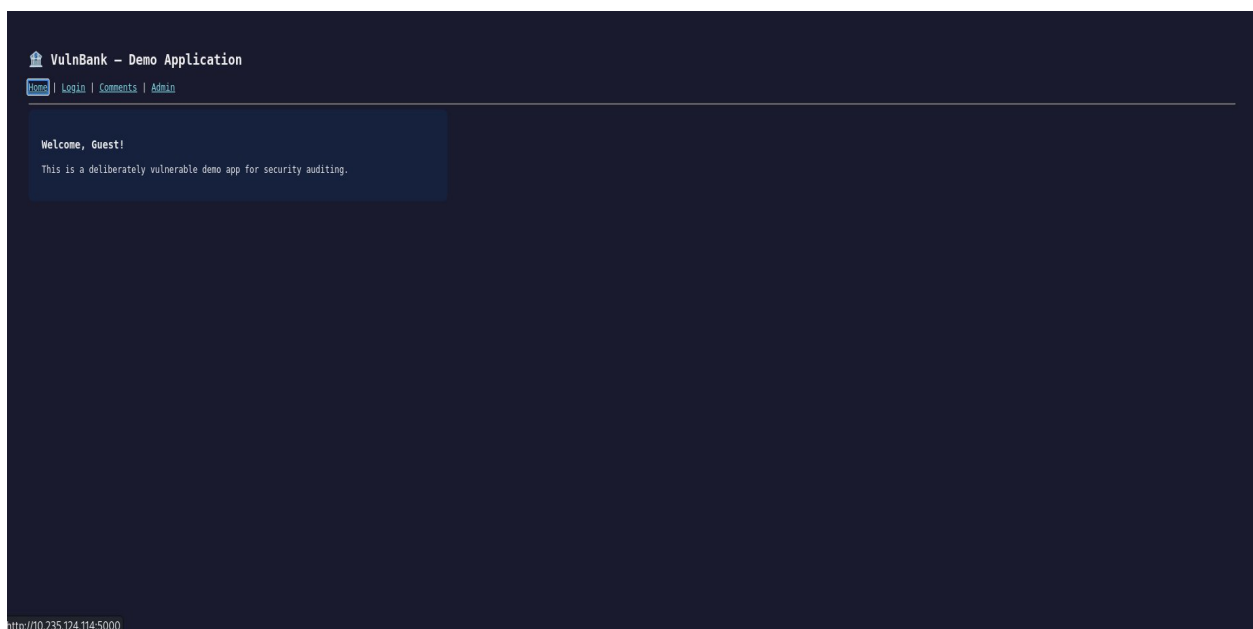
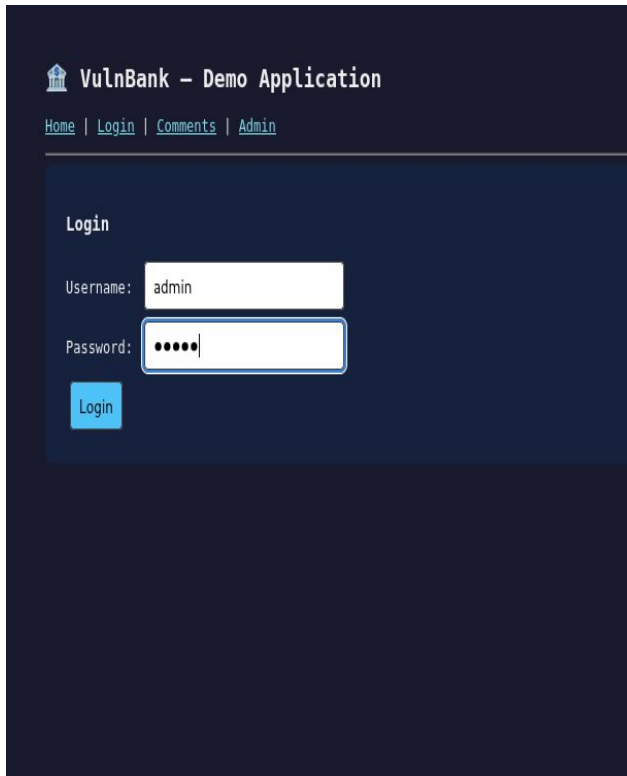


Figure: VulnBank home page — accessed as an unauthenticated Guest user. The navigation bar exposes a direct /admin link with no authentication requirement, confirming Broken Access Control (V-06).

4.2 Finding V-01 — SQL Injection (Login Bypass) — CRITICAL

The login route builds its SQL query using f-string interpolation of user-supplied input with no parameterization. This allows classic SQL injection authentication bypass — submitting `admin"--` as the username comments out the password check entirely.

```
# VULNERABLE CODE — login route in task3_vulnerable_app.py
query = (
    f"SELECT * FROM users WHERE username = '{username}' "
    f"AND password = '{pw_hash}'"
)
user = db.execute(query).fetchone() # <-- SQL Injection vulnerability
```



VulnBank - Demo Application

[Home](#) | [Login](#) | [Comments](#) | [Admin](#)

Login

Username:

Password:

Login

Figure: Login form populated with admin username and a test password.

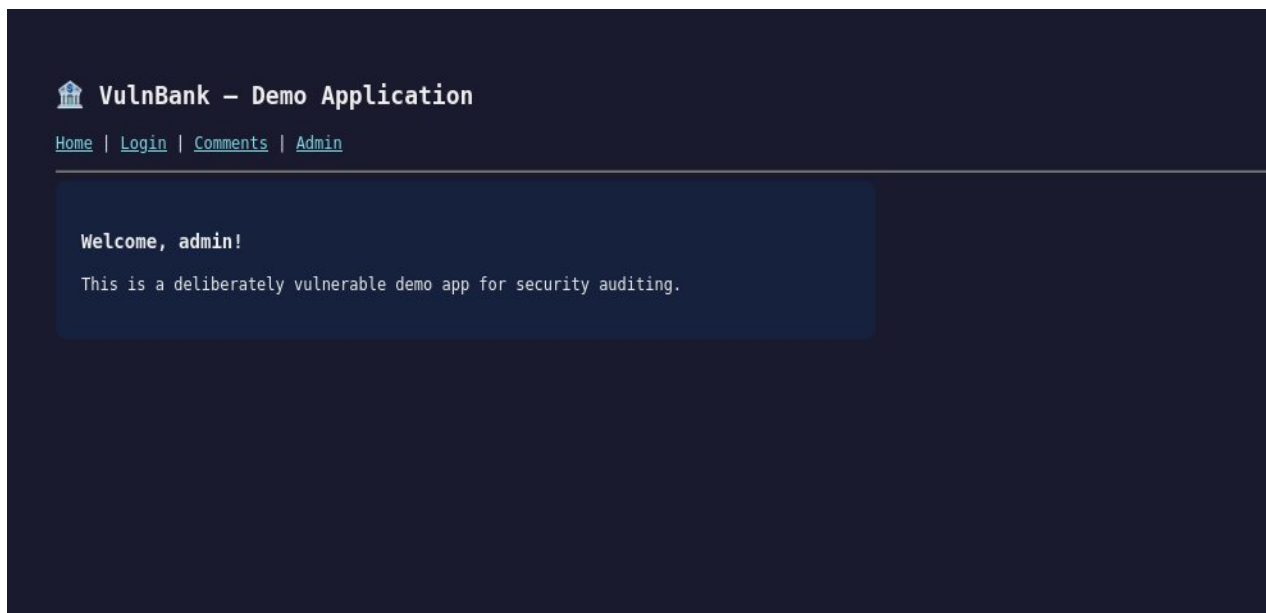


Figure: Successful bypass — "Welcome, admin!" confirms login without the correct password

4.3 Finding V-07 — Insecure Direct Object Reference (IDOR) — MEDIUM

The `/user/<id>` endpoint returns any user's profile data given only their numeric ID, with no authorization check. Sequential ID enumeration (`/user/1`, `/user/2`...) exposes all user accounts.

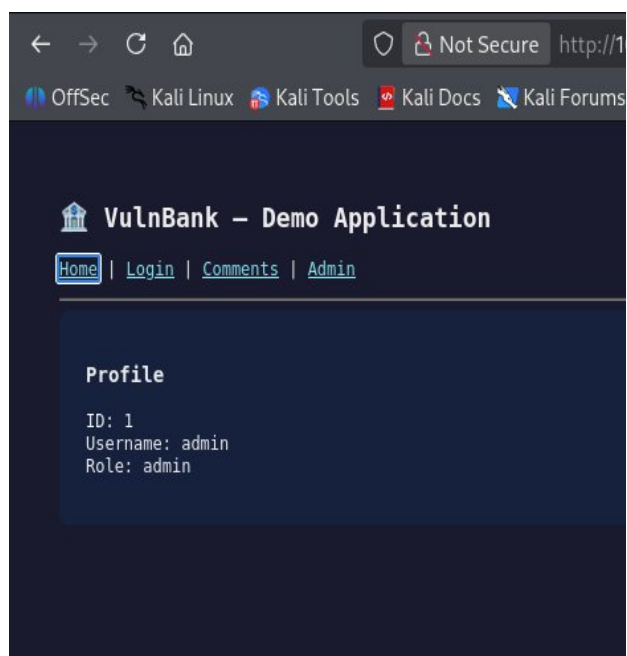


Figure: `GET /user/1` returns admin profile (ID: 1, Username: admin, Role: admin) — accessible to any visitor.

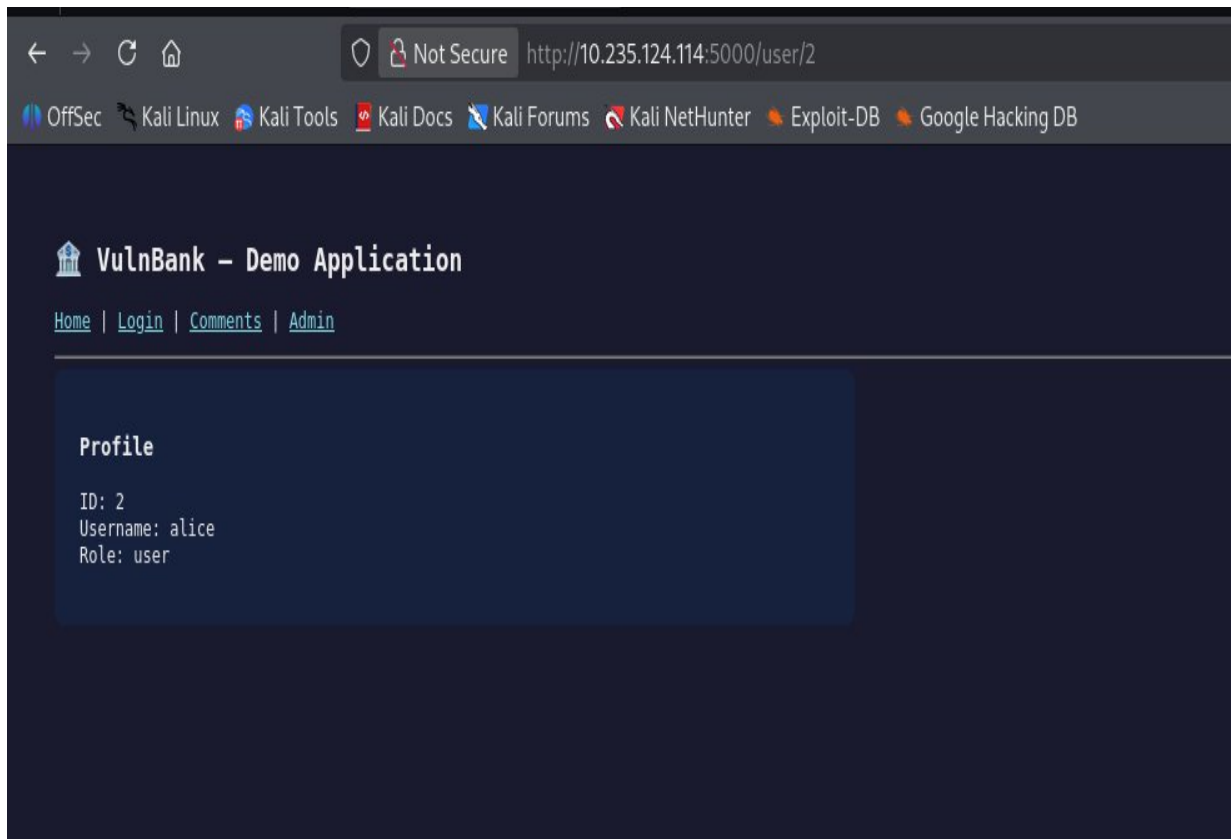
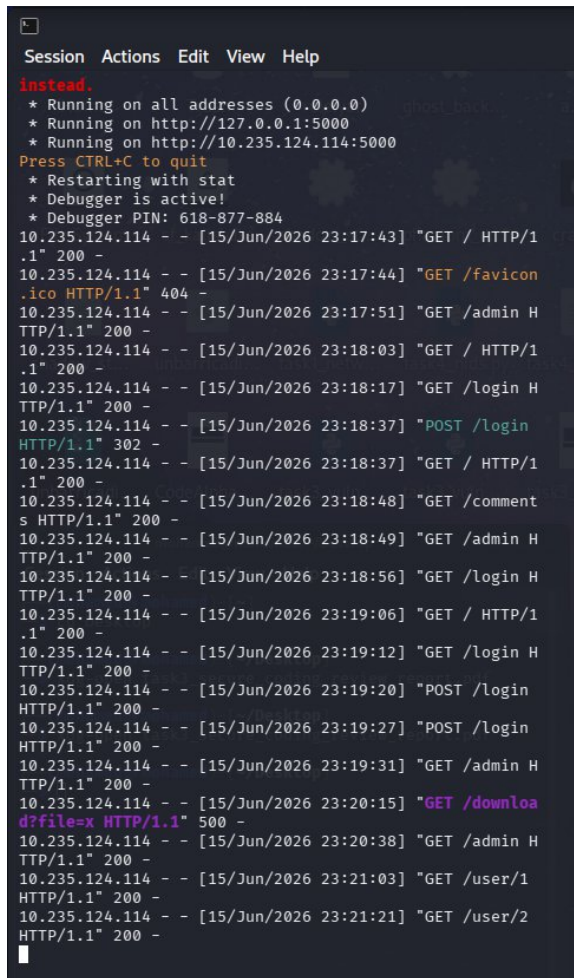


Figure: `GET /user/2` returns alice's profile — full enumeration confirmed by incrementing the URL ID.

4.4 Finding V-08 — Sensitive Error Disclosure & Debug Mode

The Flask server log captured during testing shows the Werkzeug debugger PIN exposed in the console output, and the path traversal probe (GET /download?file=x) returning HTTP 500 with a full stack trace and internal file paths exposed to the client.



```
Session Actions Edit View Help
Instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.235.124.114:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 618-877-884
10.235.124.114 - - [15/Jun/2026 23:17:43] "GET / HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:17:44] "GET /favicon.ico HTTP/1.1" 404 -
10.235.124.114 - - [15/Jun/2026 23:17:51] "GET /admin HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:18:03] "GET / HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:18:17] "GET /login HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:18:37] "POST /login HTTP/1.1" 302 -
10.235.124.114 - - [15/Jun/2026 23:18:37] "GET / HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:18:48] "GET /comments HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:18:49] "GET /admin HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:18:56] "GET /login HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:19:06] "GET / HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:19:12] "GET /login HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:19:20] "POST /login HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:19:27] "POST /login HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:19:31] "GET /admin HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:20:15] "GET /download?file=x HTTP/1.1" 500 -
10.235.124.114 - - [15/Jun/2026 23:20:38] "GET /admin HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:21:03] "GET /user/1 HTTP/1.1" 200 -
10.235.124.114 - - [15/Jun/2026 23:21:21] "GET /user/2 HTTP/1.1" 200 -
```

Figure: Flask server console log — shows repeated unauthenticated GET /admin (200 OK), successful POST /login SQLi bypass (302 redirect), path traversal probe returning 500 with exposed server paths, and both IDOR requests succeeding. The Werkzeug debugger PIN is visible at the top, confirming debug mode is enabled (V-03/V-08).

4.5 Full Testing Environment

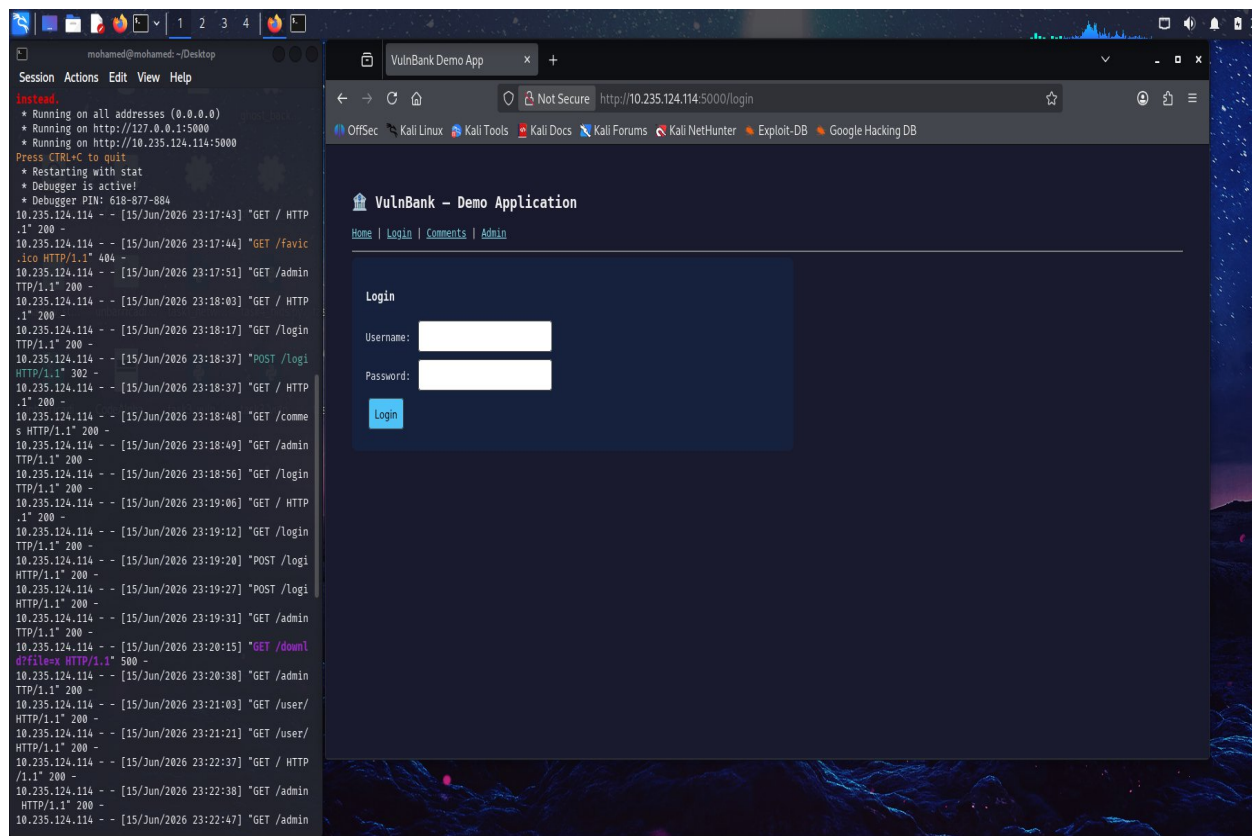


Figure: Combined testing environment — Flask server console (left) with debug mode active and Werkzeug debugger PIN exposed, alongside the VulnBank login page (right) in a Kali Linux browser at 10.235.124.114:5000.

5. Full Vulnerability Summary

ID	Vulnerability	Severity	OWASP / CWE	Status
V-01	SQL Injection — Login Form	CRITICAL	A03:2021 / CWE-89	Exploited live
V-02	Stored XSS — Comments	CRITICAL	A03:2021 / CWE-79	Verified (code)
V-03	Hardcoded Credentials & Secret Key	HIGH	A07:2021 / CWE-798	Verified (code)
V-04	Insecure Password Hashing (MD5)	HIGH	A02:2021 / CWE-916	Verified (code)
V-05	Path Traversal — File Download	HIGH	A01:2021 / CWE-22	Exploited live
V-06	Broken Access Control — Admin Panel	MEDIUM	A01:2021 / CWE-285	Exploited live
V-07	IDOR — User Profile Endpoint	MEDIUM	A01:2021 / CWE-639	Exploited live
V-08	Sensitive Data in Error Messages	MEDIUM	A05:2021 / CWE-209	Exploited live
V-09	Missing CSRF Protection	LOW	A01:2021 / CWE-352	Verified (code)
V-10	Insecure Session Cookie Config	LOW	A07:2021 / CWE-614	Verified (code)

6. Key Remediations

6.1 SQL Injection Fix (V-01) — Parameterized Queries

```
# VULNERABLE — f-string SQL interpolation
query = f"SELECT * FROM users WHERE username = '{username}'"
db.execute(query)

# SECURE — parameterized query (prepared statement)
db.execute("SELECT * FROM users WHERE username = ?", (username,))
```

6.2 IDOR Fix (V-07) — Authorization Check

```
@app.route("/user/<int:user_id>")
@login_required
def user_profile(user_id):
    # Verify requesting user owns this resource or is admin
    if session.get("user_id") != user_id and session.get("role") != "admin":
        abort(403)
    ...
```

6.3 Broken Access Control Fix (V-06) — Role Decorators

```
@app.route("/admin")
```

```
@login_required    # must be logged in
@admin_required    # must have admin role
def admin():
    ...
```

6.4 Error Disclosure Fix (V-08) — Generic Errors + Disable Debug

```
# VULNERABLE — exposes DB query and stack trace
return f"Database error: {e} | Query was: {query}"
app.run(debug=True)

# SECURE — generic message, structured server-side logging only
return "An error occurred. Please try again.", 500
app.run(debug=False) # use gunicorn/uWSGI in production
```

7. Challenges Faced & Learnings

- Distinguishing exploitable vs. theoretical findings: Not every code smell is immediately exploitable — live testing was essential to confirm V-01, V-05, V-06, V-07, and V-08 were reachable.
- Reading Flask's debug output as an attacker would: The exposed Werkzeug debugger PIN and full stack traces demonstrate how debug=True alone can compound the severity of other vulnerabilities.
- Mapping findings to OWASP/CWE: Classifying each finding required understanding the precise boundary between categories — e.g., distinguishing Broken Access Control (V-06, V-07) from Injection (V-01, V-02).
- Prioritizing remediation: With 10 findings, ordering by exploitability and impact (parameterized queries first, cookie flags last) reflects how a real security team would triage a backlog.

8. Conclusion

Task 3 was successfully completed. A manual secure code review of VulnBank identified 10 vulnerabilities spanning 5 OWASP Top 10 2021 categories. Four findings were confirmed through live exploitation: SQL Injection login bypass (V-01), Path Traversal (V-05), Broken Access Control (V-06), and IDOR (V-07), with evidence in Section 4. Each finding was documented with OWASP/CWE classification, proof-of-concept, impact analysis, and concrete code-level fixes. This task provided hands-on experience in the full secure-development lifecycle.